

Applications of Statistical Modeling:

Module #1, Lecture #2

Software for Fitting Multilevel Models

Instructor: Brady T. West
Email: bwest@umich.edu

Lecture Overview

- Today, you will see an overview of popular general-purpose statistical software alternatives for fitting multilevel models
- We will consider software for both linear and generalized linear mixed-effects models
- Conceptual material from Lecture #1 is important for understanding the various software options and alternatives; we'll keep both sets of slides open simultaneously!

Lecture Overview, cont'd

- This lecture will introduce the syntax for fitting these models in various software packages
- Next week, we will apply these procedures to a real data set in an investigation of **interviewer effects**, and explore the output generated by the procedures
- **You should start practicing with the use of these procedures, preferably with a data set in mind for the final project**

R Software

- The most readily available general-purpose statistical software (because it's free)
- Lots of good tutorials available and new menu-driven interfaces (e.g., RStudio)
- Primary multilevel modeling functions:
 - `lmer()` and `glmer()` (in the **lme4** package)
 - `lme()` (in the **nlme** package; part of base install)
 - Others [e.g., `hglm()`]

R Software, cont'd

- The `lme()` function is good for fitting models to normal outcomes, and offers more options
- The `glmer()` and `lmer()` functions are good for fitting models to non-normal outcomes and models with ***crossed*** random effects; but there aren't as many options (and possibly no p-values...)
- We will focus on the `g/lmer()` functions because they handle a more general set of models; some testing is made possible by also downloading the contributed **`lmerTest`** package

R: Required Data Structure

- The procedures in R generally require a long or “vertical” data structure, with multiple rows per cluster in the data set, and ID codes
- Example (could be Cluster / ID instead):

<u>Subject</u>	<u>Time</u>	<u>DV</u>	<u>IV1</u>	<u>IV2</u>	<u>Male</u>	
1	1		15.2	1	1	1
1	2		12.3	0	1	1
1	3		12.2	1	0	1
2	1		16.3	1	2	0 ...

R: Required Data Structure

- A given data frame object therefore needs to have a “vertical” data structure
- Easiest to import data sets from other packages using functions in the **tidyverse/foreign/haven** packages, or import delimited text files
- The `reshape` function changes data structure (e.g., wide to long)
- The following syntax assumes that a data frame object with the vertical structure (`data_n`) has been created and is available

lmer() syntax

```
> library(lmerTest)
```

Load the lmerTest package,
enabling approximate tests of
fixed-effect parameters

```
> example.model <- lmer(vsae ~ ivl + time + factor(male)  
+ (time | subject),
```

Coefficient for TIME and
intercept (by default)
randomly vary across
subjects, with UN structure

First part of model
formula: DV, and
fixed effects

```
REML = T,  
data = datan)
```

Estimation method (REML),
and data frame object

```
> summary(example.model)
```

```
> ranef(example.model)
```

```
> ranova(example.model)
```

Use the summary()
function to view
estimates, tests, and
information criteria

Generate multi-parameter tests
for fixed effects (also used for
likelihood ratio tests, where two
model fit objects are input)

Print EBLUPs for all random
coefficients

Other `lmer()` options

- Maximum Likelihood Estimation (e.g., for testing fixed effects):

`REML = F`

- Mixture-based Likelihood Ratio Tests for variance components in intercept-only models (requires `lmerTest`):

`rand(example.model)`

- Heterogeneous Residual Covariance Parameters: use `lme()` !

```
library(nlme)
```

```
examp2 <- lme(dv ~ iv1 + time + factor(male),  
             random = ~time | subject, datan, method =  
             "REML", weights = varIdent(form = ~1 | male))
```

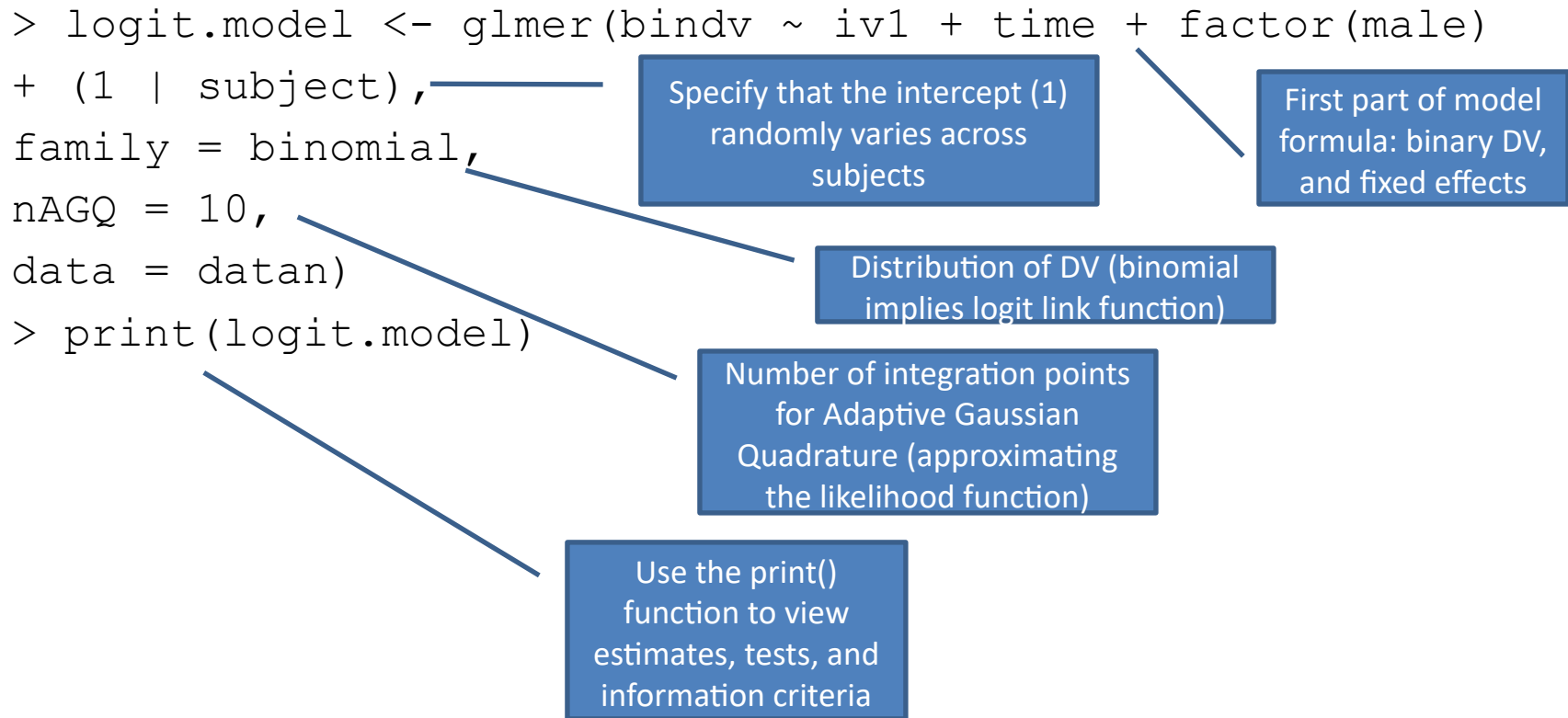
Other `lmer()` options

- Pairwise comparisons of estimated marginal means for all levels of all factors included in the model, after a model has been fitted (needs `lmerTest`):

```
> diffMeans(example.model)
```

- Model diagnostics:
 - Use `resid()` and `fitted()` functions to create new variables, and then apply various plotting functions [e.g., `plot()`] to them
 - Influence diagnostics take some work; available as part of `nlmeU` package

Logit model: `glmer()` syntax



For an excellent fully-worked example with visualization, see:

<https://stats.idre.ucla.edu/r/dae/mixed-effects-logistic-regression/>

More Details on R Procedures

- Learning the syntax is not that hard; important to consult reference manuals
- R makes many powerful graphical procedures available (see the prior worked example...)
- Some error messages can be a bit ambiguous
- The `nAGQ` argument defaults to 1, which is Laplace Approximation; higher integers = more accurate approximation = slower estimation times
- **You can generally do everything that SAS, SPSS, and Stata can do, for free; this just takes practice!**

Stata Software

- Readily available at MPSDS and JPSM
- Primary multilevel modeling procedures:
 - `mixed` (for normal outcomes)
 - `meologit`, `meologit`, etc. (for non-normal outcomes)
- **Advantages:** highly-integrated general-purpose package with powerful data management and graphical features; easy-to-use, point-and-click interface; very user-friendly and fast

Stata: Required Data Structure

- Same: Needs a “vertical” data structure
- If needed, Stata has a built-in data restructuring command (`reshape`) that is also among the best of the major statistical packages
- The following syntax assumes that a Stata data file having the vertical structure is open

mixed syntax

```
. mixed dv iv1 time i.iv2 i.male  
|| subject: time,  
covariance(unstruct)  
variance  
reml
```

First part of model
formula: DV, and fixed
effects (note the use of
i. for categorical IVs)

Second level (||): Coefficients for time and intercept (by
default) randomly vary across subjects

Print estimated **variance**
components, not standard
deviations (default)

Use unstructured
variance-covariance
matrix for random
effects

Estimation method (REML)

```
. estat ic  
. test 1.iv2 2.iv2  
. predict eblups*, reffects
```

Display information criteria

Generate multi-parameter Wald
tests for fixed effects (the
numbers are values of
categories, corresponding to
estimated fixed effects)

Save predicted values of random
effects in new variables

Other mixed options

- For maximum likelihood estimation, just leave out the `reml` argument (ML is the default)

- Heterogeneous Residual Covariance

Parameters:

```
. mixed dv iv1 time i.male || subject: time,  
covariance(unstruct) variance reml  
residuals(independent, by(male))
```

- Post-hoc pairwise comparisons of means:

```
. margins male, pwcompare(effects)
```

- After `margins`, use `marginsplot` for graphing!

Other mixed options

- Model Diagnostics:
 - Accomplished using post-estimation and plotting commands, after fitting a model using `mixed`
 - `predict st_resid, rstandard`
 - `predict predvals, fitted`
 - `twoway (scatter st_resid predvals),
yline(0)`
 - `qnorm st_resid`
 - `qnorm eblups1`
 - `qnorm eblups2`

Logit model: melogit syntax

```
. melogit bindv i.iv2 time i.male  
|| subject: ,  
covariance(identity)  
intmethod(mvaghermite)  
intpoints(7)
```

First part of model formula: DV, and fixed effects (note the use of i. for categorical IVs)

Second level (||): Only intercept is allowed to randomly vary across subjects (no variables after the colon)

Only one random effect; no need for unstructured

Approximation method (AGQ) and number of integration points; both are defaults

```
. estat ic  
. test 1.iv2 2.iv2  
. predict eblups*, reffects
```

Display information criteria

Generate multi-parameter Wald tests for fixed effects (the numbers are values of categories, corresponding to estimated fixed effects)

Save predicted values of random effects in new variables

SAS Software

- Readily available at MPSDS and JPSM, and in the Federal Statistical System
- Primary multilevel modeling procedures:
 - `proc mixed` (for normal outcomes)
 - `proc glimmix` (for non-normal outcomes)
 - `proc nlmixed` (for non-linear models)
- **Advantages:** integrated general-purpose package with powerful data management features; among most advanced procedures

SAS: Required Data Structure

- Same data structure required as in R:
 - Multiple rows per cluster (or subject)
 - Cluster / Subject ID variable
 - See Slide 6 for an example
- The next few slides assume that a SAS data set named `datan` has been created and saved in the `libn` library

proc mixed syntax

```
proc mixed data = libn.datan covtest;
```

SAS data set, and SE for
estimated covariance
parameters

```
class subject male cattime;
```

Categorical variables,
including the subject ID

```
model dv = time iv1 male / solution;
```

Dependent
variable, and fixed
effects to estimate

```
random int time / subject = subject  
type = un g gcorr v vcorr solution;
```

```
repeated cattime / subject = subject  
type = ar(1);
```

Which coefficients
should be allowed to
randomly vary across
subjects (int, time)?
What is structure of
 D matrix (type = un)?
Also display
estimated D and V
matrices, and EBLUPs

```
run;
```

For longitudinal data, indicate categorical
version of time variable (cattime), subject ID,
and type of R matrix (e.g., AR(1))

Other `proc mixed` options

- Maximum Likelihood Estimation (e.g., for testing fixed effects):

```
proc mixed data = libn.datan covtest method = ml;
```

- Heterogeneous Residual Covariance Parameters (also possible for `random`):

```
repeated cattime / subject = subject  
    type = ar(1) group = male;
```

- Post-hoc Pairwise Comparisons of Means:

```
lsmeans male / adjust = tukey;
```

Other proc mixed options

- Model Diagnostics:

```
ods graphics on;  
ods rtf file = "c:\temp\diags.rtf" style =  
    journal;  
ods exclude influence;  
proc mixed data = libn.datan plots =  
    (residualpanel boxplot influencestatpanel);  
class subject male cattime;  
model dv = time iv1 male / solution residual  
    outpred = pdat1 influence (iter = 5 effect =  
    subject est);  
ods output solutionR = eblupsdat ;
```

proc glimmix syntax

```
proc glimmix data = libn.datan;
```

SAS data set

```
class subject male;
```

Categorical variables,
including the subject ID

```
model bindv (event = "1") = time iv1 male /  
  dist = binary link = logit  
  solution cl;
```

Binary dependent variable,
fixed effects to estimate,
distribution of DV, and link
function (logit model)

```
random int / subject = subject solution;
```

```
covtest glm;
```

Appropriate likelihood
ratio tests for variance
components based on
mixtures of chi-square
distributions

Which coefficients
should be allowed to
randomly vary across
subjects (int)? Also
compute EBLUPs

```
run;
```


More details on SAS procedures

- SAS Online Help (Best Resource!)
- West, Welch, and Galecki (2022)
- *SAS for Mixed Models* (Littell et al.)
- Several alternative estimation procedures are available for `proc glimmix` (pseudo-likelihood, ML with Laplace Approximation, ML with Adaptive Quadrature); more on this choice later in lecture

So What Estimation Method Should Be Used for GLMMs?

- As shown in the previous slides, there are many possible methods for estimating the parameters in GLMMs
- When using Adaptive Gaussian Quadrature (AGQ) to approximate the likelihood (probably the most popular method), more integration points means better approximation, but also more computation time

So What Estimation Method Should Be Used for GLMMs?

- Kim et al. (2013, *TAS*; see Canvas) suggest that Laplace approximation (# integration points = 1) works well, but AGQ with more points will be better for smaller sample sizes (e.g., longitudinal data)
- Laplace is far superior to AGQ when working with large sample sizes, but can be biased
- Pseudo-likelihood / penalized quasi-likelihood estimation can lead to considerable bias!

Other Software Options

- MLwiN, HLM (multilevel specification)
- SuperMix, aML
- ASReml, SAS `proc hpmixed` (for large data sets)
- Mostly boutique packages that do not also offer data management capabilities
- Stata, `glimmix`, HLM, and MLwiN currently offer correct theoretical options for **weighted estimation** of multilevel models: more on this in two weeks
- For R users working with weights: the relatively new **svylme** package

Bayesian Estimation

- Stan: <https://mc-stan.org/users/interfaces/>
- R packages: RStanArm; brms (see https://websites.umich.edu/~bwest/brms_example.txt for an example)
- A new Multilevel Regression and Poststratification (MRP) interface: <https://mrpinterface.shinyapps.io/shinymrp/>

Thinking Ahead...

- Now that we have discussed conceptual background and software for fitting multilevel models, we will consider a case study next week, focusing on estimation of interviewer variance in the ESS Data
- Please continue practicing with these procedures using your own data! We will have time for more questions next week.